Déjà Vu

One of the most unsatisfying aspects of a video game today is that every other foe you conquer bears a boring resemblance to the one you just dispatched, that every path you walk down seems somehow familiar, and that the ecosystem of the world you are eager to save is anaemically limited to the same flora and fauna. It almost makes you want to stop the butchery. Almost!

This sameness is an unfortunate side-effect of the current API structure. Today's game needs to run to the API, which in turn runs to the driver, which then talks to the hardware, which finally complies and renders a tree. At each step, the API adds overhead—as much as 40 per cent of the entire cycle is taken up by it. Add more than one unique tree to a scene and the overhead adds up as well. Pretty soon, you would be staring at a slideshow of trees, instead of a smoothly-flowing game world. This is why game developers adopt the copy + paste method of rendering one or two different types of tree, and then use the same models throughout the land, more or less.

DX10 will reduce the API overhead by half. This would give the game more time to talk with the GPU and the rest of the system. This means that a game can now pack more content

Reduced API overhead will allow a game developer to add more detail to a title. Seen here, a screenshot from the game *Crysis2*

This method is also adopted for enemies, blades of grass, and so on.

DX10 will reduce the API overhead by half. This would give the game more time to talk with the GPU and the rest of the system. This means that a game can now pack more content, because the system has more time for the game. So the first thing you should expect from a DX10 game is a more detailed world. Will we see an army comprising unique soldiers in a future chapter of the *Total War* series? Perhaps not, but a jungle will certainly sport more than one species of tree.

Doing A 360

Microsoft and ATI jointly worked on designing the Xbox 360, and are also two prominent members of the design team for DirectX 10. It is thus no surprise that DX10 borrows some design philosophies from the Xbox 360. The most important aspect of the next-generation API is the concept of the unified shader architecture.

A quick primer: a vertex shader is a bunch of algorithms that manipulate the vertices of a triangle, which in turn make up any 3D object. Similarly, pixel shaders are software routines that manipulate individual pixels in a 3D scene. So while vertex shaders might work together to render a car, pixel shaders would colour the car, add smoke effects, rainfall, and so forth.

Traditionally, a graphics chipset carries banks of both pixel and vertex shaders. Based on a scene, the API puts each of these banks to work. Herein lies the problem. Imagine a scene that has a character standing against the sky—the pixel and vertex shaders would be equally needed to render it. Now the character steps inside a car, increasing the workload on the vertex shaders while the pixel shaders enjoy some time off. Let's say the car blows up in the next scene: lots of explosive effects—smoke, fire, debris, and so on. The pixel shaders are now needed more than the vertex shaders, which can now sit idle.

What if a card has only four pixel shaders and 12 vertex shaders? What if a scene then requires processing beyond the 12 vertex shaders' capacity? What if a scene is instead unusually high on the number of pixels pushed? All these scenarios would of course lead to game slowdown: almost all of us have noted an otherwise smoothly running game chug to below 10 frames per second at the trigger of an explosion. Here's why: game cards have dedicated resources to pixel and vertex tasks; if the game exceeds the allocated capacity, everything comes down to a terrible frame rate. More damning is the wasted silicon—why are the pixel shaders twiddling their thumbs when the vertex shaders could use some help?

A unified approach to shaders hopes to solve these problems. A graphics card with a unified shader bank can act as a pixel shader and as a vertex shader, allocating shader resources to the game according to its requirements. In theory, this should make games much faster.

In practice: programmers of today will have to unlearn their habits of programming for contemporary architectures before they can truly make use of a unified architecture—a process that will certainly take time. Note that just because DX10 has a unified shader architecture, all DX10 cards need not have unified shaders. In fact, at least for the foreseeable future, NVIDIA will stick to a contemporary architecture, whereas ATI will go the unified route. It will be interesting to see which of these two philosophies have greater merit.

Apart from a unified architecture, DX10